WNDRVR
SUPPORT NETWORK

VXWORKS 7 USB TARGET STACK PROGRAMMER'S GUIDE, SR0650

Copyright Notice

Copyright © 2022 Wind River Systems, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written permission of Wind River Systems, Inc.

Wind River, Simics, Tornado, and VxWorks are registered trademarks of Wind River Systems, Inc. Helix, Pulsar, Rocket, Titanium Cloud, Titanium Control, Titanium Core, Titanium Edge, Titanium Edge SX, Titanium Server, and the Wind River logo are trademarks of Wind River Systems, Inc. Any third-party trademarks referenced are the property of their respective owners. For further information regarding Wind River trademarks, please see:

www.windriver.com/company/terms/trademark.html

This product may include software licensed to Wind River by third parties. Relevant notices (if any) are provided for your product on the Wind River download and installation portals:

https://delivers.windriver.com/

https://windshare.usa.windriver.com/

Wind River may refer to third-party documentation by listing publications or providing links to third-party websites for informational purposes. Wind River accepts no responsibility for the information provided in such third-party documentation.

Corporate Headquarters

Wind River
500 Wind River Way
Alameda, CA 94501-1153
U.S.A.
Toll free (U.S.A.): +1-800-545-WIND
Telephone: +1-510-748-4100
Facsimile: +1-510-749-2010

For additional contact information, see the Wind River website:

www.windriver.com

For information on how to contact Customer Support, see:

www.windriver.com/support

VxWorks 7 USB Target Stack Programmer's Guide, SR0650

10 June 2020

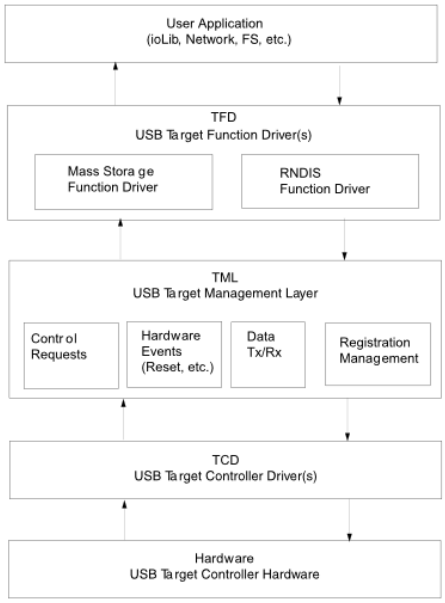
1. USB TARGET STACK

1. Overview of the USB Target Stack

The VxWorks® 7 USB Target Stack Programmer's Guide describes the use of the VxWorks USB target stack, which is the software component in the USB peripheral that interprets and responds to the commands sent by the USB host.

The figure below is a simplified representation of the architecture of the USB target stack, showing the interaction between the different layers.

Figure 1. USB Target Stack



TCD

The USB target controller driver (TCD), which must be registered with the target management layer (TML).

TML

The USB target management layer (TML), which contains the process of all the control requests and hardware events, and the transfer the data flow between the TCD and TFL.

TFL

The USB target function driver layer (TFL)., which contains different independent function drivers. To use a function driver, register it with the TML, using the correct TCD name, which was configured to attach to it.

User Application

The user application is above the TFD layer. Different function driver may use different applications.

2. Create the VxWorks Source Build for the Target Stack

To add support for the USB target stack to VxWorks, you must first create a VxWorks Source Build (VSB), which allows you to build the appropriate VxWorks kernel libraries.

Procedure

1. Create a VSB project.

   Create a VSB project based on a BSP or a CPU architecture. For general instructions on how to build a VSB project, see the VxWorks 7 Configuration and Build Guide.

2. Include the following required configuration options in your VSB.

   Enable following packages. For more information on each package, see the VxWorks Source Build Options for USB Support.

   - **USB_CORE**
   - **USB_TARGET**
   - **USB_TCORE**

   Now you have a VSB upon which to create your VxWorks Image Project (VIP). See Create the VxWorks Image Project for the USB Target Stack on page 2.

Next topic: Create the VxWorks Image Project for the USB Target Stack on page 2

3. Create the VxWorks Image Project for the USB Target Stack

After creating a VxWorks Source Build (VSB) with support for the the USB target stack, use that VSB to create the VIP project.

About This Task

A VIP allows you to build the VxWorks run-time kernel image based on the kernel libraries included in your VSB project.

Procedure

1. Create a VIP.

   Create a VIP based on the VSB created in Create the VxWorks Source Build for the Target Stack on page 2. For general instructions on how to build a VIP, see the VxWorks 7 Configuration and Build Guide.

2. Include the required components in the VIP.

   Add the following component to your VIP:

   - **INCLUDE_USB_TGT**

3. Set the **INCLUDE_USB_TGT** configuration parameters.

For more information on these parameters and their default values, see **Feature: Connectivity > Layer: USB > Components** in the VxWorks-7 API and Configuration Reference.

4. Include the **INCLUDE_USB_PLX_TCD_INIT** component.

   This is the initialization component for the USB and provides support for automatic initialization at startup.

5. Build the VIP.

Previous topic: Create the VxWorks Source Build for the Target Stack on page 2

2. TARGET FUNCTION DRIVERS

1. Adding USB Target Function Driver Support

To add support for the USB target function driver, you must include additional components in your VSB and VIP projects.

About This Task

For details on configuring a VxWorks image project (VIP), see the VxWorks 7 Configuration and Build Guide. For component and parameter descriptions, see **Feature: Connectivity > Layer: USB > Components** in the VxWorks-7 API and Configuration Reference.

To add support for the a USB target function driver, include the appropriate components in both your VxWorks source build (VSB) and your VxWorks image project (VIP).

Procedure

1. Create a VSB.

   Create a VSB according to the steps in Create the VxWorks Source Build for the Target Stack on page 2.

2. Add support for the USB target function driver to the VSB.

   Additionally include the USB_FUNCTION VSB configuration option.

3. Create a VIP based on the VSB.

   Create a VIP according to the steps in Create the VxWorks Image Project for the USB Target Stack on page 2

4. Include required components for various function drivers, as needed.

   See individual function driver packages. Add any of the components required for using the target function drivers and configure the associated parameters. For some function drivers, you can create more than one configuration.

5. Add support for initialization at startup.

   Additionally include the INCLUDE_USB_TGT_FUNC_INIT component to your VIP to add support for initialization at startup.

6. Build the VIP.

2. Writing a GEN2 USB Function Driver

Writing a GEN2 USB function driver comprises several steps, which involve adding code to the initialization and uninitialization routines, adding the function driver to the TML, and writing a target control driver.

The workflow for writing a GEN2 USB function driver comprises the following tasks:

1. Updating Initialization and Uninitialization Routines on page 5
2. Adding the Function Driver to the TML on page 6
3. Writing a Target Control Driver on page 9

4. Adding a Target Control Driver To the TML on page 13

1. Updating Initialization and Uninitialization Routines on page 5Adding code to the initialization and uninitialization routines is the first step in writing a GEN2 function driver. The initialization and uninitialization routines are called by the project facility.
2. Adding the Function Driver to the TML on page 6After adding code to the initialization and uninitialization routines, the next step in writing a GEN2 function driver is to add the function driver to the target management layer (TML).
3. Writing a Target Control Driver on page 9Writing a USB target control driver involves creating register and unregister routines for the driver and then adding the target control driver (TCD) to the target management layer (TML).
4. Adding a Target Control Driver To the TML on page 13When writing a USB target control driver, after creating register and unregister routines for the driver, add the target control driver (TCD) to the target management layer (TML).

2.1. Updating Initialization and Uninitialization Routines

Adding code to the initialization and uninitialization routines is the first step in writing a GEN2 function driver. The initialization and uninitialization routines are called by the project facility.

Procedure

1. Add code to the initialization and uninitialization routines.

   These routines are called by the project facility and are defined in the file *installDir* **/vxworks/vxworks-7/pkgs_v2/connectivity/usb/target/core/cfg/usrUsbTgtFuncInit.c**.

   1. Add code to the initialization routine.

      In the initialization routine, usrUsbTgtFuncInit( ), create a data structure used by the function driver and prepare other resources as needed.

   2. Add code to the initialization routine.

      In the uninitialization routine, usrUsbTgtFuncUnInit( ), uninitialize the USB target function drivers and release all resources.

2. Add the initialization routines into the function group initialization interface.

   Typically, you can add the new-created function driver (in the function drivers initialization group) into the usrUsbTgtFuncInit( ) routine, which is the main entry for all the function drivers to be initialized.

   If the function driver has some dependence or requirement, while being initialized, you can add your entry in another place as needed. Adding the entry into the usrUsbTgtFuncInit( ) routine is only one recommended way to manage all the function drivers, and does not depend on the common files such as **usrConfig.c** and **usrDepend.c**, since anyone can change them.

3. Add the uninitialization routines into the function group uninitialization interface.

   Be aware that the usrUsbTgtFuncUnInit( ) routine only uninitializes the function drivers, not the TCD and TML.

4. Add the function driver to the TML.

   This task is documented in Adding the Function Driver to the TML on page 6.

Parent topic: Writing a GEN2 USB Function Driver on page 4

Next topic: Adding the Function Driver to the TML on page 6

2.2. Adding the Function Driver to the TML

After adding code to the initialization and uninitialization routines, the next step in writing a GEN2 function driver is to add the function driver to the target management layer (TML).

Procedure

1. Create the **USBTGT_FUNC_DRIVER** data structure.

   For each function driver that you add to the TML, you must create a **USBTGT_FUNC_DRIVER** structure. Call usbTgtFuncCreate( ) to create this structure. See the API Reference: Kernel Connectivity for details.

   This function takes a pointer to a **USBTGT_FUNC_INFO** data structure, which the caller needs to fill in with the function driver's information.

```
 /*
 * USB Target Function Driver Information
 * This is used to create struct usbtgt_func_driver
 */

typedef struct usbtgt_func_info
        {
        USB_DESCR_HDR **                        ppFuncDescTable;                /* Function des
criptors table */
        USB_DESCR_HDR **                        ppHsFuncDescTable;              /* Function des
criptors table in High speed mode */
        USB_TARG_CALLBACK_TABLE *       pFuncCallbackTable;             /* Function driver call
back table */
        pVOID                                           pCallbackParam;                 /* Call
back parameter */
        pVOID                                           pFuncSpecific;                  /* Func
tion device specific data */
        char *                                          pFuncName;                              /
* Function driver name */
        char *                                          pTcdName;                               /
* Used for the TCD's name */
        UINT8                                           uFuncUnit;                              /
* Function driver unit */
        UINT8                                           uTcdUnit;                               /
* Used for the TCD's unit */
        UINT8                                           uConfigToBind;                  /* The
configuration group in */
        }USBTGT_FUNC_INFO, *pUSBTGT_FUNC_INFO;
```

   ppFuncDescTable

   The descriptors table which describes the characteristics of the function driver, interface/endpoint/function descriptors, such as interface descriptors to describe the Class/Subclass/Protocol the function driver use.

   ppHsFuncDescTable

   The descriptors table which describes the characteristics of the function driver, such as interface descriptors to describe the Class/Subclass/Protocol the function driver uses. This table describes the characteristics for the high speed.

pFuncCallbackTable

The function driver's callback routines table.

pCallbackParam

The function driver's callback routines parameter.

pFuncSpecific

The pointer used as the function driver's specific data, which can be used to identify the function driver instance.

pFuncName

The function driver's name, which can be used to indicate one function driver's instance with uFuncUnit parameter.

pTcdName

The TCD name which is to be attached. This is required, or the function driver will not know which TCD is to be attached. The target stack will use the user's configuration, or use the default value.

uFuncUnit

The function driver's unit, which can be used to indicate one function driver's instance with the pFuncName parameter.

uTcdUnit

The TCD unit number which needs to be attached. This is required, or the function driver will not know which TCD is to be attached. The target stack will use the user's configuration, or use the default value.

uConfigToBind

The configuration which the function driver expects to build in. The target stack will use the user's configuration, or use the default value.

2. Define the callback table and parameters.

Each function driver must provide a callback table and parameters to manage the USB hardware events and the requests from the USB host. The developer must implement these callback routines.

The callback parameter is used by the function driver; it can be any instance according to your needs and includes NULL.

```
typedef struct usbTargCallbackTable  /* USB_TARG_CALLBACK_TABLE */
        {
        /* device management callbacks */

        USB_TARG_MANAGEMENT_FUNC        mngmtFunc; /* management callback */

        /* Control pipe callbacks */

        USB_TARG_FEATURE_CLEAR_FUNC                             featureClear;                   /
* feature clear */
        USB_TARG_FEATURE_SET_FUNC                               featureSet;                     /
* feature set */
        USB_TARG_CONFIGURATION_GET_FUNC                 configurationGet;               /* conf
iguration get*/
        USB_TARG_CONFIGURATION_SET_FUNC                 configurationSet;               /* conf
iguration set*/
        USB_TARG_DESCRIPTOR_GET_FUNC                    descriptorGet;                  /* desc
riptor get */
        USB_TARG_DESCRIPTOR_SET_FUNC                    descriptorSet;                  /* desc
riptor set */
```


WNDRVR

```
        USB_TARG_INTERFACE_GET_FUNC                             interfaceGet;                   /
* interface get */
        USB_TARG_INTERFACE_SET_FUNC                             interfaceSet;                   /
* interface set */
        USB_TARG_STATUS_GET_FUNC                                statusGet;                      /
* status get */
        USB_TARG_ADDRESS_SET_FUNC                               addressSet;                     /
* address set */
        USB_TARG_SYNCH_FRAME_GET_FUNC                   synchFrameGet;                  /* fram
e get */
        USB_TARG_VENDOR_SPECIFIC_FUNC                   vendorSpecific;                 /* vend
or specific */
        } USB_TARG_CALLBACK_TABLE, *pUSB_TARG_CALLBACK_TABLE;
```

3. Fill in the descriptors table.

   Each USB function driver has its descriptors, which describe the class/subclass/protocol the driver follows and the endpoints characteristics that it uses. All of this information is included in the interface descriptor and the endpoint descriptor.

   The user must provide the interface, and the endpoint and function descriptors. The device descriptor and the configuration descriptor are generated automatically by the GEN2 target stack according to the user's configuration.

   Note that the function driver can also provide the device descriptor and use it, but if another function driver is also included and attached to the same TCD that works as one composite device, then using one specific function driver's device descriptor may affect other function drivers.

   For example, following is the high speed descriptor table used by MSC. It only provides the interface and the endpoint descriptors.

```
LOCAL pUSB_DESCR_HDR gMscHighSpeedDescHdr[] =
        {
        (pUSB_DESCR_HDR )&gMscIfDescr,            /* Interface descriptor */
        (pUSB_DESCR_HDR )&gMscBulkOutEpDescr,   /* Endpoint descriptor */
        (pUSB_DESCR_HDR )&gMscBulkInEpDescr,     /* Endpoint descriptor */
         NULL                                                   /* End of descriptors, must hav
e */
         };
```

   Note: NULL is included at the end of the table.

4. Define the user configuration group.

   You can configure the USB target stack so that multiple target controllers can exist on a single system, and all target controllers can act as independent functions when attached to different host ports. The functions can be different instances of the same or different function drivers. This is done by matching a given target function driver instance to an associated target controller driver instance. For example, if a board has two USB target controller hardware instances (two USB device ports), one port can be a USB mass storage device, another can be a USB serial port; or the two ports can both be USB storage device ports, or both USB serial ports.

   The matching mechanism for each group is based on four parameters: pTcdName, uTcdUnit, pFuncName, and uFuncUnit. The target stack uses these to manage the function driver. The “Driver Name” specifies the target controller or function driver to be used, the “Unit” specifies the target controller driver or function driver instances for a specific configuration. To be compatible with the target stack, when using Configuration Groups, the definition of the new function driver's components should follow the guidelines:

   - Components for Group *NUM* , use **INCLUDE_USBTGT**_*funcDrv_NUM*

- Parameters for Group *NUM* , use **USBTGT**_*funcDrv_configurationItemNUM*

This matching mechanism is implemented by each function driver as a group of components; each component in the component group is a "match" between the target function driver instance and the target controller driver instance. For example, suppose the USB Disk Emulation function driver implements a group of two components INCLUDE_USBTGT_MSC_1 and INCLUDE_USBTGT_MSC_2, which are grouped under a "selection" component. If the platform to be used has two USB target controllers (two USB device ports) and you want to use both of the ports as mass storage device when they are attached to two different host ports, then you could select two of the components, in each of them specifying the correct "matching" using the "Driver Name" and "Unit." For example, you can use **port1** for the first instance of the target mass storage function driver, and **port2** for the 2nd target mass storage function driver instance. For more information, see About Disk Emulation on page 15.

If the same component model is implemented by other target function drivers, in the previous example, you can then choose another function driver component in its component group and specifying its correct "matching" with the target controller driver instance, then you can have two ports acting as different target functions when they are attached to the host ports. If you just have one device port on the platform (which is the most possible situation on the market), just select one of the two components in the component group, and specifying the correct matching would be enough.

In the occasional case where your platform has more than two target controllers and you want to use them all as same target function, then you can just extend the component configuration to add another component, such as INCLUDE_USBTGT_MSC_3 (for which you just need to copy the existing code and adapt it accordingly). This target function driver component model allows the most complicated usage cases while still maintaining its simplest case—in each component you are just configuring for the usage configuration for the target port you want to use and the other parameters in each component just represent the information specific for that target function instance.

5. Add the function driver to the TML.

   Call usbTgtFuncAdd( ) to add the function driver to the TML. See the API Reference: Kernel Connectivity **usbTgt** entry for details.

6. Write the target control driver.

   For this task, follow the steps in Writing a Target Control Driver on page 9.

Parent topic: Writing a GEN2 USB Function Driver on page 4

Previous topic: Updating Initialization and Uninitialization Routines on page 5
Next topic: Writing a Target Control Driver on page 9

2.3. Writing a Target Control Driver

Writing a USB target control driver involves creating register and unregister routines for the driver and then adding the target control driver (TCD) to the target management layer (TML).

Creating Register and Unregister Routines

About This Task

The steps for writing a target control driver follow the VxBus process to initialize the driver and register it to VxBus.

Procedure

1. Create the register routine.

   Use the following usbVxbMhdrcTcdRegister( ) routine as an example.

WNDRVR

```
/*******************************************************************************
*
* usbVxbMhdrcTcdRegister - register the MHDRC with vxBus
*
* This routine registers the MHDRC Controller Driver driver with vxBus.
* Note that this can be called early in the initialization sequence.
*
* RETURNS: N/A
*
* ERRNO: N/A
*
* \NOMANUAL
*/

void usbVxbMhdrcTcdRegister(void)
    {
    /* Do not registered multiple times */

    if (g_UsbMhdrcTcdRegistered != FALSE)
        {
        USB_MHDRC_ERR("The MHDRC TCD driver already registered\n",
                      1, 2, 3, 4, 5, 6);
        return;
        }
    /* Init the global list */

    lstInit(&g_UsbMhdrcTcdList);

    /* Register the MHDRC driver for PCI bus as underlying bus */

    if (vxbDevRegister ((DRIVER_REGISTRATION *)
        &usbVxbPciMhdrcTcdDevRegistration)
        == ERROR)
        {
        USB_MHDRC_ERR("Registering MHDRC Driver over PCI Bus\n",
                     1, 2, 3, 4, 5 ,6);

        return;
        }

    /* Register the MHDRC driver for PLB bus as underlying bus */

    if (vxbDevRegister ((DRIVER_REGISTRATION *)
        &usbVxbPlbMhdrcTcdDevRegistration)
        == ERROR)
        {
        USB_MHDRC_ERR("Registering MHDRC Driver over PLB Bus\n",
                     1, 2, 3, 4, 5 ,6);

        return;
        }

    g_UsbMhdrcTcdRegistered  = TRUE;

    /* Set the interfaces for OCD to use as a hook */

    gpUsbMhdrcTcdLoad = usbMhdrcTcdInstanceAdd;
```

```
    gpUsbMhdrcTcdUnLoad = usbMhdrcTcdInstanceRemove;
    gpUsbMhdrcTcdEnable = usbMhdrcTcdInstanceEnable;
    gpUsbMhdrcTcdDisable = usbMhdrcTcdInstanceDisable;

    return;
    }
```

2. Create the unregister routine.

   Use the following usbVxbMhdrcTcdDeregister( ) routine as an example.

```
/*******************************************************************************
*
* usbVxbMhdrcTcdDeregister - deregister the MHDRC with vxBus
*
* This routine deregisters the MHDRC with vxBus.
*
* RETURNS: OK, or ERROR if unable to deregister with vxBus
*
* ERRNO: N/A
*
* \NOMANUAL
*/

STATUS usbVxbMhdrcTcdDeregister (void)
    {
    if (g_UsbMhdrcTcdRegistered != TRUE)
        {
        USB_MHDRC_ERR("The MHDRC TCD driver already deregistered\n",
                      1, 2, 3, 4, 5, 6);
        return ERROR;
        }

    /* Deregister the MHDRC driver for PLB bus as underlying bus */

    if (vxbDriverUnregister ((DRIVER_REGISTRATION *)
         &usbVxbPlbMhdrcTcdDevRegistration) == ERROR)
        {
        USB_MHDRC_ERR("Deregistering MHDRC Driver over PLB Bus\n",
                      1, 2, 3, 4, 5 ,6);
        return ERROR;
        }

    /* Deregisterr the MHDRC driver for PCI bus as underlying bus */

    if (vxbDriverUnregister ((DRIVER_REGISTRATION *)
         &usbVxbPciMhdrcTcdDevRegistration) == ERROR)
        {
        USB_MHDRC_ERR("Deregistering MHDRC Driver over PCI Bus\n",
                      1, 2, 3, 4, 5 ,6);
        return ERROR;
        }

    g_UsbMhdrcTcdRegistered = FALSE;
    return OK;
    }
```


WNDRVR

3. Add the register routine into the TCD group register interface.

```
/*******************************************************************************
*
* vxbUsbTcdRegister - register the usb target fucntion drivers to vxBus
*
* This routine is used to register the usb target fucntion drivers to vxBus
*
* RETURNS: N/A
*
* ERRNO: N/A
*/

void vxbUsbTcdRegister
        (
        void
        )
        {

#if defined(INCLUDE_USB_MHDRC_TCD_INIT) || defined(INCLUDE_USB_MHDRC_OCD_TCD_INIT)

        usbVxbMhdrcTcdRegister();

#endif /* INCLUDE_USB_MHDRC_TCD_INIT || ... */

        return;
        }
```

4. Add the unregister routine into the TCD group unregister interface, following this example.

```
/*******************************************************************************
*
* usbTcdUnRegister - unregister the usb target fucntion drivers from vxBus
*
* This routine is used to unregister the usb target fucntion drivers from vxBus
*
* RETURNS: N/A
*
* ERRNO: N/A
*/

void usbTcdUnRegister
        (
        void
        )
        {

#if defined(INCLUDE_USB_MHDRC_TCD_INIT) || defined(INCLUDE_USB_MHDRC_OCD_TCD_INIT)

        usbVxbMhdrcTcdDeregister();

#endif /* INCLUDE_USB_MHDRC_TCD_INIT || ...*/

        return;
        }
```

5. Add the target control driver to the target management layer, as described in Adding a Target Control Driver To the TML on page 13.

Parent topic: Writing a GEN2 USB Function Driver on page 4

Previous topic: Adding the Function Driver to the TML on page 6
Next topic: Adding a Target Control Driver To the TML on page 13

2.4. Adding a Target Control Driver To the TML

When writing a USB target control driver, after creating register and unregister routines for the driver, add the target control driver (TCD) to the target management layer (TML).

Procedure

1. Create a **USBTGT_TCD_INFO** data structure and fill in its specific information/characteristics.

   You must specify all of the structure elements.

```
typedef struct USBTGT_TCD_INFO
        {
        VXB_DEVICE_ID       pDev;               /* Pointer to VxBus Device instance */
        USBTGT_TCD_FUNCS *      pTcdFuncs;                      /* Functions implemented by th
e TCD */
        pVOID                           pTcdSpecific;       /* Pointer to TCD specific structur
e */
        USBTGT_DEVICE_INFO  DeviceInfo;         /* Device information */

        UINT16                          uVendorID;                      /* VID */
        UINT16                          uProductID;                     /* PID */
        UINT16                          uBcdDevice;                     /* bcdDevice */
        char *                          pMfgString;             /* Manufacture string */
        char *                          pProdString;            /* Producture string */
        char *                          pSerialString;          /* Serial num string */
        char *                          pTcdName;                       /* TCD name */
        UINT8                           uTcdUnit;                   /* Tcd unit number */
        UINT8                           uConfigCount;           /* The configuration count */
        }USBTGT_TCD_INFO, *pUSBTGT_TCD_INFO;
```

   Normally, there is only one USB target controller on the platform. However, to flexibly configure more controllers, using configuration groups is recommended. The following parameters must be included in each Group, and are used by the target stack to manage the TCD driver.

   pDev

   The VxBus device instance.

   pTcdFuncs

   The USB target controller driver (TCD) function table, which contains some routines, such as the **submitErp** function pointer, that can be called by the TML. Each TCD must provide this function table to manage and process the requirements from the function drivers and TML.

   pTcdSpecific

   This is the TCD specific data structure pointer, used to identify the real hardware. Used by the TML to check if this TCD has been bound with the hardware or not.

WNDRVR

DeviceInfo

This data structure specifics the TCD hardware features, such as the endpoints features (supported Tx and Rx endpoints) and controller features. The developer needs to provide this information.

uVendorId/uPorductId/uBcdDevice

Each USB device have his own vendor ID, product ID, and BCD device number. Since one USB controller can be one USB device, we configure the parameters with the controller configuration.

pMfgString

The manufacturer string of the USB target.

pProdString

The product string of the USB target.

pSerialString

The serial string of the USB target.

pTcdName

The TCD name to identify the hardware.

uTcdUnit

The TCD unit number to identify the hardware.

uConfigCount

The configuration count which is supported by the TCD driver.

2. Handling more than one controller.

   Normally, there is only one USB target controller on the platform. Thus, you do not need to use the configuration group for TCD. However, for greater flexibility in configuring more than on controller, we strongly recommend using configuration groups.

   The following list of the above parameters must be included in each Group, which is used by the target stack to manage the TCD driver.

```
UINT16 uVendorID; /* VID */
UINT16 uProductID;    /* PID */
UINT16 uBcdDevice;    /* bcdDevice */
char * pMfgString;    /* Manufacture string */
char * pProdString;   /* Producture string */
char * pSerialString; /* Serial num string */
char * pTcdName;      /* TCD name */
UINT8 uTcdUnit;       /* Tcd unit number */
UINT8 uConfigCount;   /* The configuration count */
```

3. Call usbTgtTcdCreate( ).

   After creating the **USBTGT_TCD_INFO**, call usbTgtTcdCreate( ) to fill in all of the specific information in the data structure.

   This routine takes one parameter, a pointer to a **USBTGT_TCD_INFO** data structure. The caller needs to fill in the TCD driver's information in this data structure as one parameter to create one **USBTGT_TCD_INFO** data structure.

The routine usbTgtTcdCreate( ) creates the **USBTGT_TCD_INFO** data structure and adds it to the global TCD list **gUsbTgtTcdList**. It returns a valid **USBTGT_TCD_INFO** pointer if successfully created or NULL if there is an error.

4. Add the TCD to the TML.

   Call usbTgtTcdAdd( ) to add the TCD to the TML.

   This routine takes one argument, which is the a pointer to the **USBTGT_TCD_INFO** data structure. The routine checks for any function drivers that need to attach to it with usbTgtFuncAttach( ). If it finds them, it attaches the function drivers onto it and generates the device characteristics, such as the device descriptor and configuration descriptor, and make it ready for use.

Parent topic: Writing a GEN2 USB Function Driver on page 4

Previous topic: Writing a Target Control Driver on page 9

3. About Disk Emulation

Disk emulation provides mechanisms for the user to statically and dynamically configure the storage devices to be visible to the connected host.

The Disk Emulation driver enables on-board memory devices, such as ram disk, NAND, and NOR flash, and plug-in memory devices, such as SD cards and USB sticks, to act as disk emulators. Disk emulation can use also specific physical file systems, such as DosFs and HRFS, or it can work as raw block devices. Disk emulation enables communication of USB hosts to a standard SCSI-like disk command set specified in USB storage media class specification. USB disk emulation also supports more than one LUN (Logic Unit Number). For most function drivers, there are two USB disk emulation configuration components, the format for which is described in Adding the Function Driver to the TML on page 6. Each component allows you to create a configuration using the configuration parameters. The disk emulation driver is implemented using the User Configuration Group model.

3. TARGET CONTROLLER DRIVERS

1. Adding Support for the PLX Target Controller Driver to VxWorks

To add support for the USB PLX target controller driver, you must include additional components in your VSB and VIP projects.

About This Task

Wind River USB provides support for the PLX target control driver (TCD) for the PLX serial USB controllers. These controllers include:

- PLX USB 3382, the PCIe USB 3.0 target controller
- PLX USB 3380, the PCIe USB 3.0 target controller
- PLX USB2380, the PCIe USB 2.0 target controller
- PLX Netchip 2282, the PCI USB 2.0 target controller

For details on configuring a VxWorks imaFor details on configuring a VxWorks image project (VIP)ge project (VIP), see the *VxWorks 7 Configuration and Build Guide*. For component and parameter descriptions, see **Feature: Connectivity > Layer: USB > Components** in the VxWorks-7 API and Configuration Reference.

Procedure

1. Create a VSB.

   Create a VSB according to the steps in Create the VxWorks Source Build for the Target Stack on page 2.

2. Add support for the USB keyboard target function driver to the VSB.

   Additionally include the following VSB configuration options:

   - USB_FUNCTION
   - USB_PLX

3. Create a VIP based on the VSB.

   Create a VIP according to the steps in Create the VxWorks Image Project for the USB Target Stack on page 2

4. Include INCLUDE_USB_PLX_TCD.
5. Set the INCLUDE_USB_PLX_TCD configuration parameter values, as needed.

   The configuration parameters must be defined for each group that is included in your VIP. For more information on these parameters and their default values, see **Feature: Connectivity > Layer: USB > Components** in the VxWorks-7 API and Configuration Reference.

6. Include the INCLUDE_USB_PLX_TCD_INIT component.

   This component supports initialization at startup.

7. Optionally include the debugging component and set the level.

   Add the INCLUDE_USB_TRACK_PLXTCD component to include the PLX TCD track support for the USB Track utility.

   Set the desired debug level in the USB_TRACK_PLXTCD parameter. The default is 1.

WNDRVR

- 0: OFF
- 1: ERROR
- 2: WARNING
- 3: NORMAL
- 4: VERBOSE

8. Build the VIP.

2. Adding Support for the Quark Target Controller Driver to VxWorks

To add support for the USB Quark target controller driver, you must include additional components in your VSB and VIP projects.

About This Task

The Quark USB target controller driver (TCD) is used for USB target functionality to emulate as a mass storage disk. This driver is used for the Galileo board. The package name for this driver is **usb_ctlr_pchudc**.

For details on configuring a VxWorks image project (VIP), see the VxWorks 7 Configuration and Build Guide. For component and parameter descriptions, see **Feature: Connectivity > Layer: USB > Components** in the VxWorks-7 API and Configuration Reference.

Procedure

1. Create a VSB.

   Create a VSB according to the steps in Create the VxWorks Source Build for the Target Stack on page 2.

2. Add support for the USB keyboard target function driver to the VSB.

   Additionally include the following VSB configuration options:

   - USB_FUNCTION
   - USB_PCHUDC

3. Create a VIP based on the VSB.

   Create a VIP according to the steps in Create the VxWorks Image Project for the USB Target Stack on page 2

4. Include INCLUDE_USB_PCHUDC_TCD_INIT.
5. Set the INCLUDE_USB_PCHUDC_TCD_INIT configuration parameter values, as needed.

   The configuration parameters must be defined for each group that is included in your VIP. For more information on these parameters and their default values, see **Feature: Connectivity > Layer: USB > Components** in the VxWorks-7 API and Configuration Reference.

   USB_PCHUDC_TCD_NAME

   Quark TCD controller specific name. The default value is "/usbTgt".

   USB_PCHUDC_TCD_PRODUCT_ID

   Quark TCD controller specific product ID in the USB device descriptor. The default value is {0xFF00}.

USB_PCHUDC_TCD_VENDOR_ID

Quark TCD controller specific vendor ID in the USB device descriptor. The default value is {0x1439}.

USB_PCHUDC_TCD_MFG_STRING

Quark TCD controller specific manufacture string to describe the device information. The default value is {"Wind River Systems"}.

USB_PCHUDC_TCD_PROD_STRING

Quark TCD controller specific product string to describe the device information. The default value is {"USB PCHUDC Target Device"}.

USB_PCHUDC_TCD_SERIAL_STRING

Quark TCD controller specific serial number string to describe the device information. The default value is {"0123456789012"}.

USB_PCHUDC_TCD_BCD_DEVICE

Quark TCD controller BCD device number in the USB device descriptor. The default value is {0x2000}.

6. Include the INCLUDE_USB_PLX_TCD_INIT component.

   This component supports initialization at startup.

7. Optionally include the debugging component and set the level.

   Add the INCLUDE_USB_TRACK_PCHUDC component to include support for the USB Track utility.

   Set the desired debug level in the USB_TRACK_PCHUDC parameter. The default is 1.

   - 0: OFF
   - 1: ERROR
   - 2: WARNING
   - 3: NORMAL
   - 4: VERBOSE

8. Build the VIP.

4. KEYBOARD FUNCTION DRIVERS

1. Adding USB Keyboard Target Function Driver Support

To add support for the USB keyboard target function driver, you must include additional components in your VSB and VIP projects.

About This Task

For details on configuring a VxWorks image project (VIP), see the VxWorks 7 Configuration and Build Guide. For component and parameter descriptions, **Feature: Connectivity > Layer: USB > Components** in the VxWorks-7 API and Configuration Reference.

To add support for the a USB keyboard target function driver, include the appropriate components in both your VxWorks source build (VSB) and your VxWorks image project (VIP).

Procedure

1. Create a VSB.

   Create a VSB according to the steps in Create the VxWorks Source Build for the Target Stack on page 2.

2. Add support for the USB keyboard target function driver to the VSB.

   Additionally include the following VSB configuration options:

   - USB_FUNCTION
   - USB_KBD

3. Create a VIP based on the VSB.

   Create a VIP according to the steps in Create the VxWorks Image Project for the USB Target Stack on page 2

4. Include INCLUDE_USBTGT_KBD.
5. Include required components for keyboard function drivers, as needed.

   Wind River USB supports two configuration groups to use the keyboard function driver. Each group configures one keyboard function device target to one USB target controller.

   Users typically include only Group #1, since there is usually only one USB target controller on the platform. Each component defines one configuration group. Add one or both components, depending upon your system needs:

   - INCLUDE_USBTGT_KBD_1
   - INCLUDE_USBTGT_KBD_2

6. Set the INCLUDE_USBTGT_KBD_*n* configuration parameter values, as needed.

   The configuration parameters must be defined for each group that is included in your VIP. For more information on these parameters and their default values, see **Feature: Connectivity > Layer: USB > Components** in the VxWorks-7 API and Configuration Reference.

7. Include the INCLUDE_USBTGT_KBD_INIT component.

   This component supports initialization at startup.

8. Build the VIP.

5. MASS STORAGE FUNCTION DRIVERS

1. Adding USB Mass Storage Target Function Driver Support

To add support for the USB mass storage target function driver, you must include additional components in your VSB and VIP projects.

About This Task

For details on configuring a VxWorks image project (VIP), see the VxWorks 7 Configuration and Build Guide. For component and parameter descriptions, see **Feature: Connectivity > Layer: USB > Components** in the VxWorks-7 API and Configuration Reference.

To add support for the a USB mass storage target function driver, include the appropriate components in both your VxWorks source build (VSB) and your VxWorks image project (VIP).

Procedure

1. Create a VSB.

   Create a VSB according to the steps in Create the VxWorks Source Build for the Target Stack on page 2.

2. Add support for the USB mass storage target function driver to the VSB.

   Additionally include the following VSB configuration options:

   - USB_FUNCTION
   - USB_MSC

3. Create a VIP based on the VSB.

   Create a VIP according to the steps in Create the VxWorks Image Project for the USB Target Stack on page 2

4. Include INCLUDE_USBTGT_MSC.
5. Include required components for mass storage function drivers, as needed.

   Wind River USB supports two configuration groups to use the mass storage function driver. Each group configures one mass storage function device target to one USB target controller.

   Users typically include only Group #1, since there is usually only one USB target controller on the platform. Each component defines one configuration group. Add one or both components, depending upon your system needs:

   - INCLUDE_USBTGT_MSC_1
   - INCLUDE_USBTGT_MSC_2

6. Set the INCLUDE_USBTGT_MSC_*n* configuration parameter values, as needed.

   The configuration parameters must be defined for each group that is included in your VIP. For more information on these parameters and their default values, see **Feature: Connectivity > Layer: USB > Components** in the VxWorks-7 API and Configuration Reference.

USBTGT_MSC_ATTACH_DEVICE_NAME*n*

Specifies the name list of the attached storage medium device to be attached to the USB disk emulation driver. Use double quotes in the name list and separate each name with a comma (,). For example: "/ffx0:1,/ffx0:2,/ffx0:3". After the storage medium device to be attached is installed in the VxWorks system, you can execute the devs command to get its name.

For example: If Datalight FLASHFX is the attached device, formatted it into 3 partitions, executing the devs command in the target shell displays the following output:

```
There are three Datalight FLASHFX devices listed: "/ffx0: 1", "/ffx0: 2", "/ffx0: 3".
```

To use the whole flash as the storage medium device to be attached, set the device as "/ffx0" or "/ffx0: 0". To use one of the partitions as the storage medium device to be attached, set the desired partition name as the name, for example "/ffx0: 1".

This is the most important configuration parameter for the USB disk emulation driver. You must configure a valid device name list before it can be used. Each entry in the configured names list is reported as a LUN to the host (if the specified device does not exist, a LUN with no media will be reported for it).

All the parameters for this component are actually a list of attributes for the LUNs as specified by the attached storage medium device name list. Each entry in these lists should have a 1-1 mapping with the attached storage medium device name list.

These parameters specify the initial LUNs to be attached to the USB disk emulation driver at compile time. You could still add or delete LUNs at runtime through user callable APIs provided by the USB disk emulation driver. The maximum number of LUNs that can be supported is limited to be 16, according to the USB mass storage class specification. Any LUNs added beyond that limit will be ignored.

USBTGT_MSC_ATTACH_DEVICE_READONLY*n*

Specifies the read-only attribute of the attached device, where y means the attached device is read-only and n means it is readable and writable. For more than one device, use commas (,) to separate the attributes, which follow the device's order in the attached device name list.

USBTGT_MSC_ATTACH_DEVICE_REMOVABLE*n*

Specifies the removable attribute of the attached device, where y means the attached device can be removed and n means it cannot be removed. For more than one device, use commas (,) to separate the attributes, which follow the device's order.

Note: This attribute can only be set to n to indicate it is un-removable only if the target device is a USB disk on a Windows platform (Windows XP or Windows 7).

USBTGT_MSC_ATTACH_TCD_NAME*n*

Specifies the name of the target controller device to be attached.

USBTGT_MSC_ATTACH_TCD_UNIT*n*

Specifies the unit number of the target controller device to be attached.

USBTGT_MSC_PROTOCOL*n*

Specifies the command protocol of the attached device used for transfer. Currently, only the RBC (Reduced Block Commands) command protocol is supported.You should keep the default value **RBC**.

USBTGT_MSC_CONFIG_NUM*n*

Specifies the USB disk emulation's configuration number.

USBTGT_MSC_NAME*n*

This is the name of the USB disk emulation driver.

USBTGT_MSC_UNIT*n*

Specifies the unit number of the USB disk emulation driver.

7. Include the INCLUDE_USBTGT_MSC_INIT component.

   This component supports initialization at startup.

8. Build the VIP.

6. NETWORK FUNCTION DRIVERS

1. Configuring a RNDIS Driver Setup on Windows XP

To configure a RNDIS driver setup on Windows XP, follow these steps.

Procedure

1. Create a VSB.

   Create a VSB according to the steps in Create the VxWorks Source Build for the Target Stack on page 2.

2. Select **USBTGT_USE_FUNC_DEVICE_DESC**.

   Additionally include the following VSB configuration options:

   - USB_FUNCTION
   - USB_NET

3. Create a VIP based on the VSB.

   Create a VIP according to the steps in Create the VxWorks Image Project for the USB Target Stack on page 2

4. Include INCLUDE_USBTGT_RNDIS.
5. Include required components for RNDIS function drivers, as needed.

   Wind River USB supports two configuration groups to use the RNDIS function driver. Each group configures one RNDIS function device target to one USB target controller.

   For example, Group #1 can be configured as one USB RNDIS network Adapter, which connects to the NIC on the platform which named as sme0, and target to the TCD which named as "/usbTgt" with unit number "0".

   Group #2 can be configured as one USB RNDIS virtual END device, which connects to the virtual END device "usbEnd0", and target to the TCD which named as "/usbTgt"with unit number "1".

   Users typically include only Group #1, since there is usually only one USB target controller on the platform. Each component defines one configuration group. Add one or both components, depending upon your system needs:

   - INCLUDE_USBTGT_RNDIS_1
   - INCLUDE_USBTGT_RNDIS_2

6. Set the INCLUDE_USBTGT_RNDIS_*n* configuration parameter values, as needed.

   The configuration parameters must be defined for each group that is included in your VIP. For more information on these parameters and their default values, see **Feature: Connectivity > Layer: USB > Components** in the VxWorks-7 API and Configuration Reference.

7. Add support for the medium type for the Group #1 RNDIS function driver.

   RNDIS is only one interface of USB; thus, it requires an actual medium that is used to communicate with the network. If you do not include a medium type, the compiler generates a warning.

   1. The medium type choices are a device bridge or a virtual END device. Choose one for Group #1:
      - INCLUDE_USBTGT_RNDIS_MEDIUM_TYPE_END_BRIDGE1

- INCLUDE_USBTGT_RNDIS_MEDIUM_TYPE_VIRTUAL_END1

2. Set the configuration parameter values for the selected medium type component.

   The RNDIS driver uses the associated name and unit configuration parameters, which are defined for the medium type component, to find and connect to the device. For more information on these parameters and their default values, see **Feature: Connectivity > Layer: USB > Components** in the VxWorks-7 API and Configuration Reference.

8. If needed, add support for the medium type for the Group #2 RNDIS function driver.
   1. If you included support for a second RNDIS function driver, choose a medium type for that interface:
      - INCLUDE_USBTGT_RNDIS_MEDIUM_TYPE_END_BRIDGE2
      - INCLUDE_USBTGT_RNDIS_MEDIUM_TYPE_VIRTUAL_END2
   2. If you added a second RNDIS driver, set the configuration parameter values for the selected medium type component for that interface.

      The RNDIS driver uses the associated name and unit configuration parameters, which are defined for the medium type component, to find and connect to the device. For more information on these parameters and their default values, see **Feature: Connectivity > Layer: USB > Components** in the VxWorks-7 API and Configuration Reference.

   If you have included INCLUDE_USBTGT_RNDIS_MEDIUM_TYPE_END_BRIDGE1, name the END bridge device **sme0**. Define this name using the USBTGT_RNDIS_MEDIUM_END_BRIDGE_NAME1 and USBTGT_RNDIS_MEDIUM_END_BRIDGE_UNIT1 parameters. After the configuration, the RNDIS connects the network device on the platform named as **sme0** to USB. When the USB interface connects to the Windows machine, the Windows machine will find one USB network attached. The Windows machine can access the network and the internet using the **sme0** device. **sme0** is the LAN9x18 device on the Omap3530, but the **sme0** will not be used on the Omap3530 platform, itself, instead it is borrowing the NIC hardware from the Omap3530 and providing it to the Windows machine.

9. Configure additional configuration parameters, if a virtual END device was selected.

   If INCLUDE_USBTGT_RNDIS_MEDIUM_TYPE_VIRTUAL_END1 or INCLUDE_USBTGT_RNDIS_MEDIUM_TYPE_VIRTUAL_END2 was selected as the medium type for the RNDIS function driver, the INCLUDE_USBTGT_VRTL_END component is added automatically. In such cases, set the configuration parameter values for the INCLUDE_USBTGT_VRTL_END component.

   For more information on these parameters and their default values, see **Feature: Connectivity > Layer: USB > Components** in the VxWorks-7 API and Configuration Reference.

   If setting more than one local or remote END address, use double quotes in the address list and separate each address with a comma. Examples are:

   - {“00:11:22:33:44:00”,“00:22:22:33:44:00”}
   - {“00:11:22:33:55:00”,“00:22:22:33:55:00”}

   The USBTGT_VRTL_END_TYPE specifies the type of END device; however, currently, only the VxBus END driver is supported.

   0

   legacy END driver (not supported)

   1

   VxBus END driver (supported)

   2

   VxBus END2 driver (not supported)

   If you have included INCLUDE_USBTGT_RNDIS_MEDIUM_TYPE_VIRTUAL_END1, name the virtual END device on the platform **usbEnd0**. Define this name using the USBTGT_RNDIS_MEDIUM_VRTL_END_NAME1 and

WNDRVR

USBTGT_RNDIS_MEDIUM_VRTL_END_UNIT1 parameters. Then, configure IP addresses on the networks. Using this configuration, find one network on VxWorks named as **usbEnd0** and another network on Windows. After configuring different IP addresses on each side, communication can be started. The result is the same as creating two network devices and giving one to the platform and one to the Windows machine.

10. Include the INCLUDE_USBTGT_RNDIS_INIT component.

    This component supports initialization at startup and automatically includes support for initialization dependencies, such as initialization of the USB target network.

11. Set the USB target network configuration parameter values.

    The USB RNDIS function driver works as one USB target network device. Thus, including a USB RNDIS component automatically includes the INCLUDE_USBTGT_NETWORK component, which uses several data pools to store the data between USB and the network. Values for the data pools are set using the configuration parameters for the INCLUDE_USBTGT_NETWORK component. For more information on these parameters and their default values, see **Feature: Connectivity > Layer: USB > Components** in the VxWorks-7 API and Configuration Reference.

12. Build the VIP.

2. Adding USB RNDIS Ethernet Function Driver Support

To add support for the USB RNDIS function driver, you must include additional components in your VSB and VIP projects.

RNDIS Support

Wind River USB provides RNDIS (Remote Network Driver Interface Specification) support to emulate one virtual Ethernet device using the USB target controller.

You can enable the RNDIS function driver with the Ethernet emulation feature and attach to one USB target controller by registering to the Target Management Level (TML). For information about TML, see Adding the Function Driver to the TML on page 6). The USB controller then emulates one Ethernet device, as if it were attached to one USB host. The Ethernet device is found on both the host side and the target side, enabling the host and the target to communicate with each other using the network protocol. The VxWorks Network Stack is required for this functionality and must be included in the VxWorks image project.

Wind River USB RNDIS support also provides a device bridge between the USB controller and other kinds of hardware controllers, such as network controllers. When you connect a USB cable to the USB host, the USB host can directly access the real hardware on the target via the USB interface. The role of the USB is to act as the bridge between the USB host and the real hardware. Note that the actual hardware device on the target is not usable by the target, since it has been handed over to the host by USB.

Configuring VxWorks for RNDIS

About This Task

For details on configuring a VxWorks image project (VIP), see the VxWorks 7 Configuration and Build Guide. For component and parameter descriptions, see **Feature: Connectivity > Layer: USB > Components** in the VxWorks-7 API and Configuration Reference.

To add support for the a USB RNDIS function driver, include the appropriate components in both your VxWorks source build (VSB) and your VxWorks image project (VIP).

Procedure

1. Create a VSB.

   Create a VSB according to the steps in Create the VxWorks Source Build for the Target Stack on page 2.

2. Add support for the USB RNDIS function driver to the VSB.

   Additionally include the following VSB configuration options:

   - USB_FUNCTION
   - USB_NET

3. Create a VIP based on the VSB.

   Create a VIP according to the steps in Create the VxWorks Image Project for the USB Target Stack on page 2

4. Include INCLUDE_USBTGT_RNDIS.
5. Include required components for RNDIS function drivers, as needed.

   Wind River USB supports two configuration groups to use the RNDIS function driver. Each group configures one RNDIS function device target to one USB target controller.

   For example, Group #1 can be configured as one USB RNDIS network Adapter, which connects to the NIC on the platform which named as sme0, and target to the TCD which named as "/usbTgt" with unit number "0".

   Group #2 can be configured as one USB RNDIS virtual END device, which connects to the virtual END device "usbEnd0", and target to the TCD which named as "/usbTgt"with unit number "1".

   Users typically include only Group #1, since there is usually only one USB target controller on the platform. Each component defines one configuration group. Add one or both components, depending upon your system needs:

   - INCLUDE_USBTGT_RNDIS_1
   - INCLUDE_USBTGT_RNDIS_2

6. Set the INCLUDE_USBTGT_RNDIS_*n* configuration parameter values, as needed.

   The configuration parameters must be defined for each group that is included in your VIP. For more information on these parameters and their default values, see **Feature: Connectivity > Layer: USB > Components** in the VxWorks-7 API and Configuration Reference.

7. Add support for the medium type for the Group #1 RNDIS function driver.

   RNDIS is only one interface of USB; thus, it requires an actual medium that is used to communicate with the network. If you do not include a medium type, the compiler generates a warning.

   1. The medium type choices are a device bridge or a virtual END device. Choose one for Group #1:
      - INCLUDE_USBTGT_RNDIS_MEDIUM_TYPE_END_BRIDGE1
      - INCLUDE_USBTGT_RNDIS_MEDIUM_TYPE_VIRTUAL_END1
   2. Set the configuration parameter values for the selected medium type component.

   The RNDIS driver uses the associated name and unit configuration parameters, which are defined for the medium type component, to find and connect to the device. For more information on these parameters and their default values, see **Feature: Connectivity > Layer: USB > Components** in the VxWorks-7 API and Configuration Reference.

8. If needed, add support for the medium type for the Group #2 RNDIS function driver.
   1. If you included support for a second RNDIS function driver, choose a medium type for that interface:

- INCLUDE_USBTGT_RNDIS_MEDIUM_TYPE_END_BRIDGE2
- INCLUDE_USBTGT_RNDIS_MEDIUM_TYPE_VIRTUAL_END2

2. If you added a second RNDIS driver, set the configuration parameter values for the selected medium type component for that interface.

The RNDIS driver uses the associated name and unit configuration parameters, which are defined for the medium type component, to find and connect to the device. For more information on these parameters and their default values, see **Feature: Connectivity > Layer: USB > Components** in the VxWorks-7 API and Configuration Reference.

If you have included INCLUDE_USBTGT_RNDIS_MEDIUM_TYPE_END_BRIDGE1, name the END bridge device **sme0**. Define this name using the USBTGT_RNDIS_MEDIUM_END_BRIDGE_NAME1 and USBTGT_RNDIS_MEDIUM_END_BRIDGE_UNIT1 parameters. After the configuration, the RNDIS connects the network device on the platform named as **sme0** to USB. When the USB interface connects to the Windows machine, the Windows machine will find one USB network attached. The Windows machine can access the network and the internet using the **sme0** device. **sme0** is the LAN9x18 device on the Omap3530, but the **sme0** will not be used on the Omap3530 platform, itself, instead it is borrowing the NIC hardware from the Omap3530 and providing it to the Windows machine.

9. Configure additional configuration parameters, if a virtual END device was selected.

   If INCLUDE_USBTGT_RNDIS_MEDIUM_TYPE_VIRTUAL_END1 or INCLUDE_USBTGT_RNDIS_MEDIUM_TYPE_VIRTUAL_END2 was selected as the medium type for the RNDIS function driver, the INCLUDE_USBTGT_VRTL_END component is added automatically. In such cases, set the configuration parameter values for the INCLUDE_USBTGT_VRTL_END component.

   For more information on these parameters and their default values, see **Feature: Connectivity > Layer: USB > Components** in the VxWorks-7 API and Configuration Reference.

   If setting more than one local or remote END address, use double quotes in the address list and separate each address with a comma. Examples are:

   - {“00:11:22:33:44:00”,“00:22:22:33:44:00”}
   - {“00:11:22:33:55:00”,“00:22:22:33:55:00”}

   The USBTGT_VRTL_END_TYPE specifies the type of END device; however, currently, only the VxBus END driver is supported.

   0

   legacy END driver (not supported)

   1

   VxBus END driver (supported)

   2

   VxBus END2 driver (not supported)

   If you have included INCLUDE_USBTGT_RNDIS_MEDIUM_TYPE_VIRTUAL_END1, name the virtual END device on the platform **usbEnd0**. Define this name using the USBTGT_RNDIS_MEDIUM_VRTL_END_NAME1 and USBTGT_RNDIS_MEDIUM_VRTL_END_UNIT1 parameters. Then, configure IP addresses on the networks. Using this configuration, find one network on VxWorks named as **usbEnd0** and another network on Windows. After configuring different IP addresses on each side, communication can be started. The result is the same as creating two network devices and giving one to the platform and one to the Windows machine.

10. Include the INCLUDE_USBTGT_RNDIS_INIT component.

This component supports initialization at startup and automatically includes support for initialization dependencies, such as initialization of the USB target network.

11. Set the USB target network configuration parameter values.

    The USB RNDIS function driver works as one USB target network device. Thus, including a USB RNDIS component automatically includes the INCLUDE_USBTGT_NETWORK component, which uses several data pools to store the data between USB and the network. Values for the data pools are set using the configuration parameters for the INCLUDE_USBTGT_NETWORK component. For more information on these parameters and their default values, see **Feature: Connectivity > Layer: USB > Components** in the VxWorks-7 API and Configuration Reference.

12. Set the USB target device product ID to **0xFF01**.

    Set the parameter USB_FSL_TCD_PRODUCT_ID to **0xFF01** when adding the USB RNDIS Ethernet function driver.

13. Build the VIP.

7. PRINTER FUNCTION DRIVERS

1. Adding USB Printer Target Function Driver Support

To add support for the USB printer target function driver, you must include additional components in your VSB and VIP projects.

About This Task

For details on configuring a VxWorks image project (VIP), see the VxWorks 7 Configuration and Build Guide. For component and parameter descriptions, see **Feature: Connectivity > Layer: USB > Components** in the VxWorks-7 API and Configuration Reference.

To add support for the a USB printer target function driver, include the appropriate components in both your VxWorks source build (VSB) and your VxWorks image project (VIP).

Procedure

1. Create a VSB.

   Create a VSB according to the steps in Create the VxWorks Source Build for the Target Stack on page 2.

2. Add support for the USB printer target function driver to the VSB.

   Additionally include the following VSB configuration options:

   - USB_FUNCTION
   - USB_PRNT

3. Create a VIP based on the VSB.

   Create a VIP according to the steps in Create the VxWorks Image Project for the USB Target Stack on page 2

4. Include INCLUDE_USBTGT_PRN.
5. Include required components for printer function drivers, as needed.

   Wind River USB supports two configuration groups to use the printer function driver. Each group configures one printer function device target to one USB target controller.

   Users typically include only Group #1, since there is usually only one USB target controller on the platform. Each component defines one configuration group. Add one or both components, depending upon your system needs:

   - INCLUDE_USBTGT_PRN_1
   - INCLUDE_USBTGT_PRN_2

6. Set the INCLUDE_USBTGT_PRN_*n* configuration parameter values, as needed.

   The configuration parameters must be defined for each group that is included in your VIP. For more information on these parameters and their default values, see **Feature: Connectivity > Layer: USB > Components** in the VxWorks-7 API and Configuration Reference.

7. Include the INCLUDE_USBTGT_PRN_INIT component.

   This component supports initialization at startup.

8. Build the VIP.

8. SERIAL FUNCTION DRIVERS

1. Adding USB Serial Target Function Driver Support

To add support for the USB serial target function driver, you must include additional components in your VSB and VIP projects.

About This Task

For details on configuring a VxWorks image project (VIP), see the VxWorks 7 Configuration and Build Guide. For component and parameter descriptions, see **Feature: Connectivity > Layer: USB > Components** in the VxWorks-7 API and Configuration Reference.

To add support for the a USB serial target function driver, include the appropriate components in both your VxWorks source build (VSB) and your VxWorks image project (VIP).

Procedure

1. Create a VSB.

   Create a VSB according to the steps in Create the VxWorks Source Build for the Target Stack on page 2.

2. Add support for the USB serial target function driver to the VSB.

   Additionally include the following VSB configuration options:

   - USB_FUNCTION
   - USB_SER

3. Create a VIP based on the VSB.

   Create a VIP according to the steps in Create the VxWorks Image Project for the USB Target Stack on page 2

4. Include INCLUDE_USBTGT_SER.
5. Set the INCLUDE_USBTGT_SER configuration parameter values, as needed.

   The configuration parameters must be defined for each group that is included in your VIP. For more information on these parameters and their default values, see **Feature: Connectivity > Layer: USB > Components** in the VxWorks-7 API and Configuration Reference.

6. Include the INCLUDE_USBTGT_SER_INIT component.

   This component supports initialization at startup.

7. Set the USB target device product ID to **0xFF02**.

   Set the parameter USB_FSL_TCD_PRODUCT_ID to **0xFF02** when adding the USB Serial target function driver.

8. Build the VIP.